Cargar datos: de la fuente al DataFrame

Clase teórico-práctica

AUTOR/A
Joselina Davyt-Colo

0.1 Qué cubre esta clase y qué no

El anexo de errores clasifica los mensajes que produce una lectura fallida. El práctico B ejercita el caso del CSV mal formado. Esta clase cubre lo anterior a las dos cosas: **cuántas fuentes distintas existen, qué decide cada función de lectura y qué información sobre los datos sobrevive según de dónde vengan**.

Ningún bloque de este documento se ejecuta al compilar: las salidas están escritas debajo de cada bloque, como en el material de fundamentos. Para reproducirlas hay que correr una vez `generar_datos_clase.py`, que crea la carpeta `datos_clase/` con los archivos de ejemplo en los seis formatos.

> ⓘ **Sobre los tipos que se ven abajo**
>
> Las columnas de texto aparecen como `object` en pandas 2 y como `str` en pandas 3. Es el mismo tipo con dos nombres; acá se usa `object`, que es lo que trae la instalación de Anaconda de la mayoría.

1 Parte I. Teoría

1.1 Un archivo no es una tabla

Un archivo es una secuencia de bytes. Una tabla es una estructura con filas, columnas y un tipo por columna. Entre una cosa y la otra hay una traducción, y esa traducción no es única: el mismo archivo admite varias lecturas, todas sintácticamente válidas y solo una correcta.

Esto no es una limitación técnica que un software mejor podría resolver. Es una propiedad del problema. El archivo no contiene la instrucción de cómo leerse a sí mismo, así que la instrucción tiene que venir de otro lado: de quien lo escribió, de la documentación, o —lo más frecuente— de una hipótesis que uno formula y después verifica.

1.2 Las cinco decisiones de toda lectura

Cada vez que se ejecuta `read_csv()` o su equivalente, la función toma cinco decisiones. Si no se declaran, las toma por defecto, en silencio y sin avisar.

#	Decisión	Si se equivoca
1	Dónde termina un campo y empieza el siguiente	Columnas de menos, o todo en una sola
2	Qué secuencias de caracteres son números	Columnas numéricas que quedan como texto
3	Qué representa un valor ausente	Faltantes que entran a los cálculos como dato
4	Cuál fila es el encabezado	Nombres de columna absurdos y una fila perdida
5	Cómo se traduce cada byte a un carácter	Eñes y tildes rotas, o un error de codificación

Las cinco existen en todos los formatos, aunque en algunos vengan resueltas de fábrica. Esa es exactamente la diferencia entre formatos que veremos más adelante.

1.3 El ciclo: leer, inspeccionar, declarar, releer

De ahí se sigue el método de trabajo de esta clase, que es el mismo para cualquier fuente:

1. **Leer** con los argumentos por defecto. Esta primera lectura no sirve para analizar: sirve para ver qué supuso el programa.
2. **Inspeccionar** el resultado contra el archivo crudo: forma, tipos, primeras filas, faltantes.
3. **Declarar** explícitamente las decisiones que estaban mal.
4. **Releer** con esas decisiones escritas en el código.

La primera lectura es una hipótesis; la inspección la contrasta; la segunda lectura la corrige y la deja registrada. Lo que queda versionado en el repositorio no es el archivo: es la lectura declarada.

1.4 Qué puede cargar cada formato

Los formatos se distinguen por cuánta información sobre los propios datos son capaces de transportar. Cuanto menos transporta el archivo, más hay que reconstruir a mano en la lectura.

Formato	Transporta	Hay que reconstruir
CSV	Texto y nada más	Tipos, faltantes, orden de categorías, todo
Excel	Texto, algo de formato, varias hojas	Tipos reales, dónde empieza la tabla

Formato	Transporta	Hay que reconstruir
Stata `.dta`	Tipos, etiquetas de variable y de valor	Poco
SPSS `.sav`	Tipos, etiquetas, faltantes declarados	Poco
Parquet	Tipos exactos, incluido el orden de las categóricas	Nada
Base de datos	Tipos, relaciones, restricciones	Nada, pero hay que escribir la consulta
JSON de una API	Estructura anidada	El aplanado y todos los tipos

> ⓘ **Importante**
>
> Un `.csv` no es un formato peor que un `.dta`: es un formato que delega. Todo lo que el `.dta` guarda adentro, el `.csv` lo deja en manos del código que lo lee. Por eso un `.csv` sin su diccionario de variables es un dataset incompleto.

1.5 Procedencia: de dónde vino y si se puede volver a buscar

Un dato tiene, además de contenido, un origen que hay que poder declarar. No es lo mismo un archivo que alguien bajó a mano hace seis meses que una URL escrita en el código.

Origen	Se puede reproducir	Riesgo
Archivo suelto en el escritorio	No	Nadie sabe de dónde salió ni de cuándo es
Archivo en el repositorio	Sí	Se desactualiza en silencio
URL en el código	Sí, mientras la URL viva	La fuente cambia el archivo sin avisar
API con parámetros	Sí, y con la consulta explícita	Requiere credenciales y control de versión

La regla práctica que se sigue en el curso: **descargar una vez, guardar la copia con fecha, y trabajar sobre la copia**. La descarga queda en el código para que sea reproducible; el análisis no depende de que la conexión funcione hoy.

2 Parte II. Seis fuentes

Los archivos de ejemplo representan una cadena de locales comerciales con ventas mensuales por sucursal. Los crea `generar_datos_clase.py`, que hay que correr una sola vez antes de seguir.

2.1 CSV: el caso base

El CSV es el formato que más delega, y por eso el que más decisiones exige. Este archivo usa las tres convenciones habituales en datos uruguayos: punto y coma como separador, coma decimal y codificación `latin-1`.

Primera lectura: la hipótesis por defecto

```
import pandas as pd
from pathlib import Path

CARPETA = Path("datos_clase")

ingenua = pd.read_csv(CARPETA / "ventas_local.csv", encoding="latin-1")

ingenua.shape          # (6, 1)
list(ingenua.columns)  # ['id_local;sucursal;mes;ventas_uyu;empleados']
list(ingenua.index)[:2]
```

```
['101;Centro;2025-01;128500', '102;Centro;2025-02;143200']
```

Cinco columnas se convirtieron en una. Y hay algo peor que eso: la coma decimal de `128500,50` sí funcionó como separador, así que cada fila quedó partida en dos pedazos y el primero terminó usado como nombre de fila. La tabla no está incompleta, está corrida.

> ⚠ **Advertencia**
>
> Ningún mensaje de error apareció. Una lectura con el separador equivocado es, en los dos lenguajes, una falla silenciosa: devuelve un objeto que parece un DataFrame y se puede seguir trabajando con él.

Inspección

Antes de arreglar nada, mirar el archivo crudo. Son tres líneas y evitan adivinar:

```
for linea in (CARPETA / "ventas_local.csv").read_text(encoding="latin-1").splitlines()[:3]:
    print(linea)
```

```
id_local;sucursal;mes;ventas_uyu;empleados
101;Centro;2025-01;128500,5;12
102;Centro;2025-02;143200,0;12
```

Ahí se leen las tres decisiones de una: el separador es `;`, el decimal es `,`, y el encabezado está en la primera fila.

Segunda lectura: la hipótesis declarada

```
ventas = pd.read_csv(CARPETA / "ventas_local.csv",
                     sep=";", decimal=",", encoding="latin-1")
ventas
```

```
   id_local sucursal      mes  ventas_uyu  empleados
0       101   Centro  2025-01   128500.50         12
1       102   Centro  2025-02   143200.00         12
2       103  Pocitos  2025-01    98750.25          8
3       104   Cordón  2025-02   110430.75          9
4       105  Pocitos  2025-03   152980.00          8
5       106   Cordón  2025-03   121045.40          9
```

```
ventas.dtypes
```

```
id_local        int64
sucursal       object
mes            object
ventas_uyu    float64
empleados       int64
dtype: object
```

El mismo archivo, la misma función, tres argumentos de diferencia: seis filas por cinco columnas con `ventas_uyu` numérica, en lugar de una columna de texto.

En R:

```
library(readr)

# read_csv2() ya asume ";" como separador y "," como decimal
ventas <- read_csv2("datos_clase/ventas_local.csv",
                    locale = locale(encoding = "latin1"))

# la forma general, declarando cada decisión por separado
ventas <- read_delim("datos_clase/ventas_local.csv",
                     delim = ";",
                     locale = locale(decimal_mark = ",", encoding = "latin1"))
```

2.2 Excel: dónde empieza la tabla

Una planilla no es un archivo de datos: es un documento que puede contener datos. Tiene títulos, notas al pie, celdas combinadas, varias hojas y filas en blanco de adorno. La decisión número 4 —cuál fila es el encabezado— acá es la que manda.

```
pd.read_excel(CARPETA / "ventas.xlsx").head(4)
```

```
  Reporte de ventas — uso interno Unnamed: 1 Unnamed: 2  Unnamed: 3 Unnamed: 4
0          Generado el 12/03/2025        NaN        NaN         NaN        NaN
1                             NaN        NaN        NaN         NaN        NaN
2                       id_local   sucursal        mes  ventas_uyu  empleados
3                             101     Centro    2025-01    128500.5         12
```

Las tres filas de título se leyeron como si fueran datos: los nombres de columna salieron del texto del título y quedaron `Unnamed: 1`, `Unnamed: 2`. La tabla real empieza en la cuarta fila.

Un archivo de Excel además contiene varias hojas, y por defecto se lee la primera:

```
pd.ExcelFile(CARPETA / "ventas.xlsx").sheet_names
# ['ventas', 'locales']
```

Declarando la hoja y cuántas filas saltear:

```
ventas_xl = pd.read_excel(CARPETA / "ventas.xlsx", sheet_name="ventas", skiprows=3)
ventas_xl.dtypes
```

```
id_local        int64
sucursal       object
mes            object
ventas_uyu    float64
empleados       int64
dtype: object
```

> **Importante**
>
> Abrir un CSV con Excel para "ver cómo es" no es inocuo: Excel interpreta al abrir, convierte códigos con ceros a la izquierda en números, y fechas en otras fechas. Si después se guarda, esas conversiones quedan escritas en el archivo. Para inspeccionar un CSV se usa un editor de texto o el propio código.

En R:

```
library(readxl)

excel_sheets("datos_clase/ventas.xlsx")
# [1] "ventas"  "locales"

ventas_xl <- read_excel("datos_clase/ventas.xlsx",
                        sheet = "ventas",
                        skip  = 3)
```

2.3 Stata y SPSS: cuando el archivo trae sus etiquetas

Son los formatos de los paquetes estadísticos, y los que va a encontrar quien trabaje con microdatos de encuesta: la Encuesta Continua de Hogares del INE y la mayoría de las encuestas de institutos de

investigación se publican en `.dta` o `.sav`.

La diferencia con un CSV es de fondo: estos archivos guardan los tipos y además guardan **etiquetas**, que son documentación adentro del propio archivo.

```
ventas_dta = pd.read_stata(CARPETA / "ventas.dta")
ventas_dta.dtypes
```

```
id_local        int32
sucursal       object
mes            object
ventas_uyu    float64
empleados       int32
dtype: object
```

Los tipos llegaron declarados: `id_local` es entero porque en el archivo dice que es entero, no porque pandas lo haya adivinado mirando los valores.

Las etiquetas de variable se leen aparte, con el lector en modo iterador:

```
with pd.read_stata(CARPETA / "ventas.dta", iterator=True) as lector:
    etiquetas = lector.variable_labels()

etiquetas
```

```
{'id_local': '',
 'sucursal': '',
 'mes': '',
 'ventas_uyu': 'Ventas mensuales en pesos uruguayos',
 'empleados': 'Personal ocupado en el local'}
```

Eso —"Ventas mensuales en pesos uruguayos"— es exactamente lo que en el práctico A había que escribir a mano en el diccionario de variables. En un `.dta` viaja con los datos.

SPSS y las etiquetas de valor

SPSS agrega un nivel más: las **etiquetas de valor**, que traducen los códigos numéricos a texto. En una encuesta la variable llega como `1`, `2`, `3` y la etiqueta dice qué significa cada número. Pandas no lee `.sav`; hace falta `pyreadstat`, que no viene con Anaconda y se instala con `pip install pyreadstat`.

```
import pyreadstat

crudo, meta = pyreadstat.read_sav(CARPETA / "ventas.sav")
crudo[["sucursal_cod", "ventas_uyu"]].head(3)
```

```
   sucursal_cod  ventas_uyu
0           1.0   128500.50
```

```
1           1.0   143200.00
2           2.0    98750.25
```

```
meta.variable_value_labels
# {'sucursal_cod': {1.0: 'Centro', 2.0: 'Pocitos', 3.0: 'Cordón'}}

legible, _ = pyreadstat.read_sav(CARPETA / "ventas.sav", apply_value_formats=True)
legible[["sucursal_cod", "ventas_uyu"]].head(3)
```

```
  sucursal_cod  ventas_uyu
0       Centro   128500.50
1       Centro   143200.00
2      Pocitos    98750.25
```

> **Nota**
>
> Dos cosas para registrar. Primera: aplicar o no las etiquetas es una decisión, no un detalle de presentación —con los códigos se puede cruzar con otros archivos de la misma encuesta, con las etiquetas se puede leer una tabla— y lo que no conviene es cambiar de criterio a mitad del análisis. Segunda: los enteros volvieron como `1.0` y no como `1`, porque SPSS no distingue enteros de decimales.

En R, el paquete `haven` cubre los dos formatos y guarda las etiquetas como atributos de la columna:

```
library(haven)

ventas_dta <- read_dta("datos_clase/ventas.dta")
attr(ventas_dta$ventas_uyu, "label")
# [1] "Ventas mensuales en pesos uruguayos"

ventas_sav <- read_sav("datos_clase/ventas.sav")
ventas_sav$sucursal_cod <- as_factor(ventas_sav$sucursal_cod)   # aplica las etiquetas
```

2.4 Una URL es una ruta más

Las funciones de lectura aceptan una dirección web donde aceptan una ruta. No hay nada especial que aprender:

```
url = "https://ejemplo.gub.uy/datos/serie.csv"
serie = pd.read_csv(url, sep=";", decimal=",", encoding="latin-1")
```

```
serie <- readr::read_csv2("https://ejemplo.gub.uy/datos/serie.csv")
```

Lo que sí hay que aprender es qué no hacer con eso. Leer directo de la URL cada vez que se corre el análisis tiene tres problemas: el análisis deja de funcionar sin conexión, la fuente puede cambiar el archivo sin avisar, y dos corridas del mismo código en fechas distintas pueden dar resultados distintos sin que quede registro.

El patrón que resuelve las tres cosas es descargar una vez y fijar la copia:

```
from datetime import date
import urllib.request

url = "https://ejemplo.gub.uy/datos/serie.csv"
destino = CARPETA / f"serie_{date.today():%Y-%m-%d}.csv"

if not destino.exists():                    # solo la primera vez
    urllib.request.urlretrieve(url, destino)

serie = pd.read_csv(destino, sep=";", decimal=",", encoding="latin-1")
```

El nombre del archivo lleva la fecha de descarga; el código de descarga queda versionado; el análisis corre sobre la copia. La procedencia queda declarada y el resultado es reproducible.

2.5 Base de datos: la consulta es parte de la lectura

Con un archivo se lee todo y se filtra después. Con una base de datos se decide cuánto traer, y esa decisión se escribe en SQL. El módulo `sqlite3` viene con Python, así que no hay nada que instalar.

```
import sqlite3

con = sqlite3.connect(CARPETA / "comercio.db")
pd.read_sql("SELECT name FROM sqlite_master WHERE type = 'table'", con)
```

```
     name
0   ventas
1  locales
```

Traer una tabla entera es lo mismo que leer un archivo. Pero también se puede pedir que el trabajo lo haga la base y que llegue el resultado ya agregado, cruzando dos tablas:

```
consulta = """
    SELECT v.sucursal,
           l.region,
           SUM(v.ventas_uyu) AS total_ventas,
           COUNT(*)          AS meses
    FROM ventas v
    JOIN locales l ON v.sucursal = l.sucursal
    GROUP BY v.sucursal, l.region
    ORDER BY total_ventas DESC
"""
resumen = pd.read_sql(consulta, con)
con.close()
resumen
```

```
  sucursal  region  total_ventas  meses
0   Centro     Sur     271700.50      2
1  Pocitos     Sur     251730.25      2
2   Cordón  Centro     231476.15      2
```

> **Nota**
>
> Acá aparece una decisión que en un CSV no existe: **cuánto del análisis ocurre antes de que los datos lleguen**. Con tablas grandes la respuesta suele ser "bastante", porque traer diez millones de filas para después promediarlas es caro y lento. Pero cada agregación hecha en SQL es una decisión analítica que queda fuera del script de análisis, y hay que documentarla igual.

En R:

```r
library(DBI)

con <- dbConnect(RSQLite::SQLite(), "datos_clase/comercio.db")
dbListTables(con)

resumen <- dbGetQuery(con, "
    SELECT v.sucursal, SUM(v.ventas_uyu) AS total_ventas
    FROM ventas v GROUP BY v.sucursal")

dbDisconnect(con)
```

2.6 API: del JSON anidado a la tabla

Una API devuelve JSON, que es una estructura de árbol y no una tabla. La dificultad no está en la conexión sino en el aplanado: hay que decidir qué nivel del árbol corresponde a una fila.

Así se ve una respuesta típica:

```json
{
  "meta": {
    "fuente": "sistema interno",
    "actualizado": "2025-03-31"
  },
  "resultados": [
    {
      "local": {"id": 101, "sucursal": "Centro"},
      "periodo": "2025-01",
      "ventas": 128500.5
    },
    {
      "local": {"id": 103, "sucursal": "Pocitos"},
      "periodo": "2025-01",
      "ventas": 98750.25
    }
```

```
  ]
}
```

Los datos no están en la raíz: están adentro de `resultados`, y cada resultado tiene a su vez un objeto `local` anidado. Convertir la lista sin más deja el diccionario metido dentro de una celda:

```
import json

crudo = json.loads((CARPETA / "respuesta_api.json").read_text(encoding="utf-8"))
pd.DataFrame(crudo["resultados"])
```

```
                              local  periodo     ventas
0   {'id': 101, 'sucursal': 'Centro'}  2025-01  128500.50
1  {'id': 103, 'sucursal': 'Pocitos'}  2025-01   98750.25
```

La columna `local` contiene objetos, no valores. `json_normalize()` aplana el árbol y arma un nombre de columna por camino:

```
pd.json_normalize(crudo["resultados"], sep="_")
```

```
   periodo     ventas  local_id local_sucursal
0  2025-01  128500.50       101         Centro
1  2025-01   98750.25       103        Pocitos
```

La llamada real a la API:

```
import requests

respuesta = requests.get("https://api.ejemplo.uy/ventas",
                         params={"desde": "2025-01", "hasta": "2025-03"},
                         timeout=30)
respuesta.raise_for_status()          # corta acá si el servidor devolvió un error

datos = pd.json_normalize(respuesta.json()["resultados"], sep="_")
```

```
library(httr2)
library(jsonlite)

datos <- request("https://api.ejemplo.uy/ventas") |>
    req_url_query(desde = "2025-01", hasta = "2025-03") |>
    req_perform() |>
    resp_body_string() |>
    fromJSON(flatten = TRUE)
```

> ⚠ **Advertencia**
>
> `raise_for_status()` no es un detalle. Sin esa línea, un error del servidor —que suele venir con su propio JSON de error— entra al análisis como si fuera un conjunto de datos vacío o distinto, sin producir ninguna excepción.

3 Parte III. La lectura declarada, encapsulada

Las lecturas corregidas de esta clase tienen algo en común: son varias líneas con argumentos que hay que acertar, y se van a repetir en cada script del proyecto. Copiarlas es garantía de que en algún momento van a quedar desalineadas.

La forma de que la lectura sea una sola decisión tomada una vez es encerrarla en una función que devuelva el dataset ya tipado:

```
ORDEN_SUCURSAL = ["Centro", "Cordón", "Pocitos"]

def cargar_ventas(ruta=CARPETA / "ventas_local.csv"):
    """Lee el archivo de ventas con todas las decisiones declaradas."""
    datos = pd.read_csv(ruta, sep=";", decimal=",", encoding="latin-1")
    datos["sucursal"] = pd.Categorical(datos["sucursal"],
                                       categories=ORDEN_SUCURSAL, ordered=False)
    datos["mes"] = pd.PeriodIndex(datos["mes"], freq="M")
    return datos

ventas = cargar_ventas()
ventas.dtypes
```

```
id_local          int64
sucursal       category
mes           period[M]
ventas_uyu      float64
empleados         int64
dtype: object
```

```
ventas.groupby("sucursal", observed=True)["ventas_uyu"].sum()
```

```
sucursal
Centro     271700.50
Cordón     231476.15
Pocitos    251730.25
Name: ventas_uyu, dtype: float64
```

Cualquier script del proyecto hace `from carga import cargar_ventas` y obtiene el mismo dataset, con los mismos tipos, sin repetir un solo argumento. Si mañana la fuente cambia el separador, se corrige en un lugar.

3.1 Resumen

Fuente	Python	R	Lo que hay que declarar
CSV	`pd.read_csv`	`readr::read_delim`	Separador, decimal, codificación, faltantes
Excel	`pd.read_excel`	`readxl::read_excel`	Hoja y dónde empieza la tabla
Stata	`pd.read_stata`	`haven::read_dta`	Casi nada; las etiquetas se leen aparte
SPSS	`pyreadstat.read_sav`	`haven::read_sav`	Si se aplican o no las etiquetas de valor
URL	La misma función, con la dirección	Igual	Cuándo se descargó y dónde quedó la copia
Base de datos	`pd.read_sql`	`DBI::dbGetQuery`	La consulta: qué traer y qué agregar antes
API	`pd.json_normalize`	`jsonlite::fromJSON`	Qué nivel del árbol es una fila

Las tres cosas que hay que llevarse:

1. La primera lectura es una hipótesis, no un resultado. Se inspecciona antes de usarla.
2. El formato determina cuánto hay que reconstruir a mano. Un CSV no es peor que un `.dta`: es un formato que delega, y hay que hacerse cargo de lo delegado.
3. La lectura declarada se escribe una vez, en una función, y se versiona. Es parte del análisis, no un paso previo al análisis.